ELEC 475

Lab 2

Lukas Hendry (20353912)

Benjamin Syer (20357140)

October 22nd, 2026

Architectural Specifications

1.1 Base LeNet-5

Base LeNet-5 has two convolutional layers with Tanh activations and non-overlapping 2×2 average pooling, followed by three fully connected layers.

Table 1: Base LeNet-5 layer specification, parameters and receptive field

Layer	Configuration	Output shape	Params	RF
Input	—	3×64×64	—	1
Conv1 + Tanh	5×5, 6 filters, s=1, p=0	6×60×60	456	5
AvgPool1	2×2, s=2	6×30×30	—	6
Conv2 + Tanh	5×5, 16 filters, s=1, p=0	16×26×26	2,416	14
AvgPool2	2×2, s=2	16×13×13	—	16
Flatten	16 × 13 × 13	2,704	—	—
FC1 + Tanh	2,704 → 120	120	324,600	—
FC2 + Tanh	120 → 84	84	10,164	—
FC3	84 → 10	10	850	—
Total			**338,486 (0.338 M)**	

1.2 Modernized LeNet

Modernized LeNet keeps the same two convolutional and three fully connected layers as Base LeNet-5 but replaces Tanh with ReLU and average pooling with overlapping 3×3 max pooling.

Table 2: Modernized LeNet layer specification, parameters and receptive field

Layer	Configuration	Output shape	Params	RF
Input	—	3×64×64	—	1
Conv1 + ReLU	5×5, 6 filters, s=1, p=2	6×64×64	456	5
MaxPool1	3×3, s=2	6×31×31	—	7

Conv2 + ReLU	5×5, 16 filters, s=1, p=2	16×31×31	2,416	15
MaxPool2	3×3, s=2	16×15×15	—	19
Flatten	16 × 15 × 15	3,600	—	—
FC1 + ReLU	3,600 → 120	120	432,120	—
FC2 + ReLU	120 → 84	84	10,164	—
FC3	84 → 10	10	850	—
Total			**446,006 (0.446 M)**	

1.3 Full AlexNet-64

AlexNet-64 has five convolutional layers with wider channels, each followed by ReLU, with overlapping max pooling after stages 1, 2 and 5, and three fully connected layers with Dropout. Optional normalization (LRN or BatchNorm2d) can be inserted after each convolution.

Table 3: AlexNet-64 layer specification, parameters and receptive field (no normalization)

Layer	Configuration	Output shape	Params	RF
Input	—	3×64×64	—	1
Conv1 + ReLU	5×5, 64, s=2, p=2	64×32×32	4,864	5
MaxPool1	3×3, s=2	64×15×15	—	9
Conv2 + ReLU	5×5, 192, s=1, p=2	192×15×15	307,392	25
MaxPool2	3×3, s=2	192×7×7	—	33
Conv3 + ReLU	3×3, 384, s=1, p=1	384×7×7	663,936	49
Conv4 + ReLU	3×3, 384, s=1, p=1	384×7×7	1,327,488	65
Conv5 + ReLU	3×3, 256, s=1, p=1	256×7×7	884,992	81
MaxPool3	3×3, s=2	256×3×3	—	97
Flatten	256 × 3 × 3	2,304	—	—
Dropout + FC1 + ReLU	2,304 → 1,024	1,024	2,360,320	—

Dropout + FC2 + ReLU	1,024 → 1,024	1,024	1,049,600	—
FC3	1,024 → 10	10	10,250	—
Total			**6,608,842 (6.609 M)**	

1.4 PyTorch Model ZooAlexNet Baseline

Zoo AlexNet has five convolutional layers with a large 11×11 stride-4 first convolution and no normalization, followed by adaptive average pooling and three much larger fully connected layers with Dropout.

Table 4: PyTorch Model Zoo AlexNet layer specification on a 64×64 input

Layer	**Configuration**	**Output shape (64×64 input)**	**Params**	**RF**
Input	—	3×64×64	—	1
Conv1 + ReLU	11×11, 64, s=4, p=2	64×15×15	23,296	11
MaxPool1	3×3, s=2	64×7×7	—	19
Conv2 + ReLU	5×5, 192, s=1, p=2	192×7×7	307,392	51
MaxPool2	3×3, s=2	192×3×3	—	67
Conv3 + ReLU	3×3, 384, s=1, p=1	384×3×3	663,936	99
Conv4 + ReLU	3×3, 256, s=1, p=1	256×3×3	884,992	131
Conv5 + ReLU	3×3, 256, s=1, p=1	256×3×3	590,080	163
MaxPool3	3×3, s=2	256×1×1	—	195
AdaptiveAvgPool2d	output 6×6	256×6×6	—	—
Flatten	256 × 6 × 6	9,216	—	—

Dropout + FC1 + ReLU	9,216 → 4,096	4,096	37,752,832	—
Dropout + FC2 + ReLU	4,096 → 4,096	4,096	16,781,312	—
FC3	4,096 → 10	10	40,970	—
Total			**57,044,810 (57.045 M)**	

Step 2: Empirical Normalization Study

Variant A: No Normalization

Table 5: AlexNet-64 (No Normalization) model complexity

Metric	Value
Total parameters	6,608,842 (6.609 M)
Trainable parameters	6,608,842
Multiply-accumulates	218.35 MMACs
FLOPs	436.70 MFLOPs
Weight size (FP32)	25.21 MB
Forward/backward pass size	1.29 MB
Estimated total memory	27.77 MB

Table 6: AlexNet-64 (No Normalization) per-epoch results on CIFAR-10

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)	Val Macro F1 (%)	Time (s)	Throughput (samples/s)
1	1.7783	32.45	1.4349	46.18	44.62	21.59	2315.8
2	1.3600	50.56	1.1854	57.03	56.31	17.82	2806.1
3	1.1823	57.94	1.0817	61.36	61.17	16.63	3007.5
4	1.0474	63.22	1.0053	64.27	64.18	18.65	2681.5
5	0.9489	66.86	0.9176	68.13	67.82	18.00	2777.7
6	0.8839	69.31	0.9416	67.66	68.51	21.11	2368.7
7	0.8195	71.68	0.8257	71.72	71.20	35.65	1402.5
8	0.7700	73.71	0.7929	72.87	72.92	30.24	1653.3
9	0.7328	74.93	0.7947	72.70	72.67	54.82	912.1
10	0.6879	76.45	0.8013	73.33	73.33	52.09	959.9

Table 7: AlexNet-64 (No Normalization) training summary

Summary	Value
Best Val Acc	73.33% (epoch 10)
Best Val Macro F1	73.33% (epoch 10)
Final Train Acc	76.45%
Generalization gap at epoch 10	3.12 pp
Total training time	287.14 s
Mean epoch time	28.66 s
Mean throughput	2088.5 samples/s

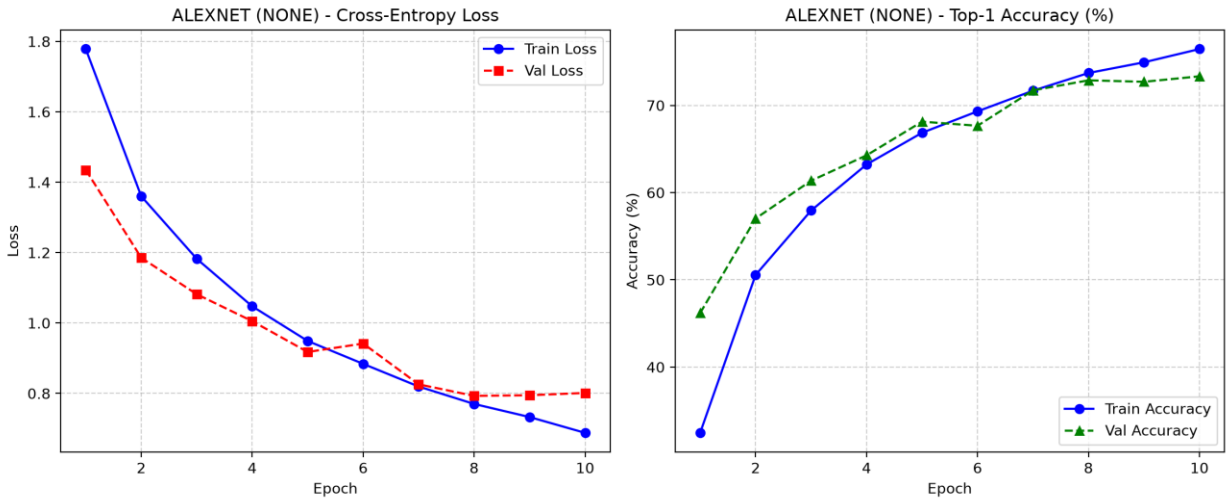


Figure 1: AlexNet-64 (No Normalization) training and validation loss and accuracy

Variant B: Local Response Normalization

Table 8: AlexNet-64 + LRN model complexity

Metric	Value
Total parameters	6,608,842 (6.609 M)
Trainable parameters	6,608,842
Multiply-accumulates	218.35 MMACs
FLOPs	436.70 MFLOPs
Weight size (FP32)	25.21 MB
Forward/backward pass size	1.29 MB
Estimated total memory	27.77 MB

Table 9: AlexNet-64 + LRN per-epoch results on CIFAR-10

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)	Val Macro F1 (%)	Time (s)	Throughput (samples/s)
1	1.8221	30.07	1.4605	44.80	42.52	28.26	1769.1
2	1.3868	49.01	1.2256	55.80	54.78	27.76	1801.0
3	1.1894	57.22	1.0806	60.33	60.26	26.87	1860.5
4	1.0641	62.26	0.9940	64.22	63.56	27.32	1830.0
5	0.9659	65.89	0.9008	68.53	68.35	28.64	1746.0
6	0.8895	68.99	0.8765	69.38	69.64	29.53	1693.4
7	0.8271	71.46	0.8364	70.96	70.99	31.06	1609.5
8	0.7652	73.49	0.8009	71.81	71.95	29.61	1688.5
9	0.7219	75.06	0.7692	73.31	73.19	22.24	2247.8
10	0.6779	76.69	0.7730	73.52	73.27	16.72	2991.2

Table 10: AlexNet-64 + LRN training summary

Summary	Value
Best Val Acc	73.52% (epoch 10)
Best Val Macro F1	73.27% (epoch 10)
Final Train Acc	76.69%
Generalization gap at epoch 10	3.17 pp
Total training time	268.50 s
Mean epoch time	26.80 s
Mean throughput	1923.7 samples/s

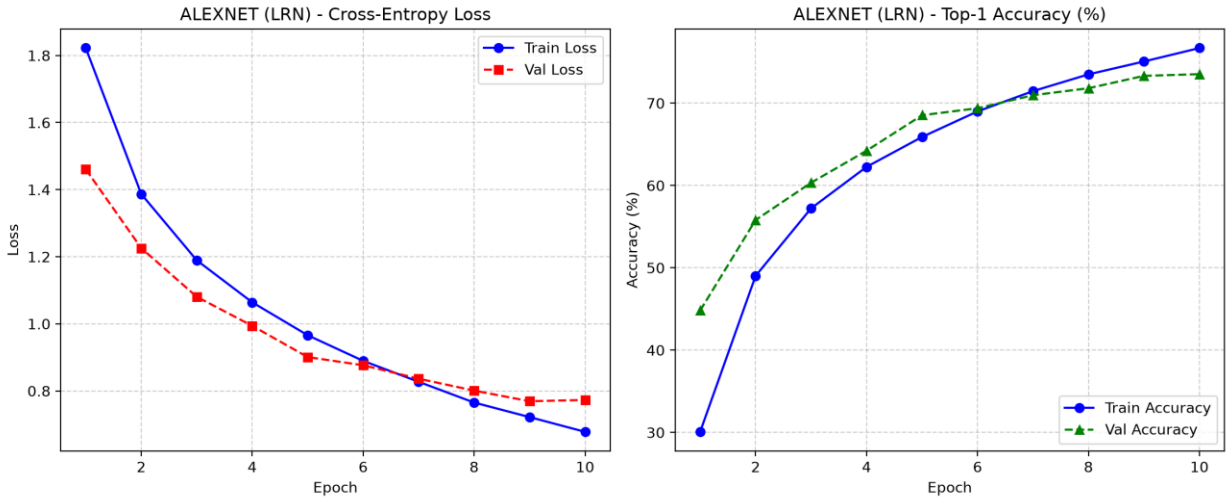


Figure 2: AlexNet-64 + LRN training and validation loss and accuracy

Variant C: Modern Batch Normalization

Table 11: AlexNet-64 + BatchNorm2d model complexity

Metric	Value
Total parameters	6,611,402 (6.611 M)
Trainable parameters	6,611,402
Multiply-accumulates	218.98 MMACs
FLOPs	437.97 MFLOPs
Weight size (FP32)	25.22 MB
Forward/backward pass size	2.56 MB
Estimated total memory	29.05 MB

Table 12: AlexNet-64 + BatchNorm2d per-epoch results on CIFAR-10

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)	Val Macro F1 (%)	Time (s)	Throughput (samples/s)
1	1.4856	44.93	1.3711	50.98	49.43	14.60	3424.0
2	1.0925	61.31	1.2801	56.64	55.10	12.79	3910.4
3	0.9209	68.01	0.9168	68.14	67.66	12.89	3880.1
4	0.7995	72.54	0.7865	72.80	72.94	12.97	3855.3
5	0.7203	75.62	0.7835	73.50	73.68	12.99	3848.9
6	0.6451	78.45	1.0140	68.20	68.36	13.05	3830.5
7	0.5848	80.34	0.7053	76.53	76.69	13.42	3726.9
8	0.5253	82.54	0.6720	77.80	77.82	13.73	3642.1
9	0.4758	84.19	0.6591	78.31	78.06	13.96	3581.0
10	0.4425	85.33	0.6219	79.84	80.28	14.06	3557.4

Table 13: AlexNet-64 + BatchNorm2d training summary

Summary	Value
Best Val Acc	79.84% (epoch 10)
Best Val Macro F1	80.28% (epoch 10)
Final Train Acc	85.33%
Generalization gap at epoch 10	5.49 pp
Total training time	134.86 s
Mean epoch time	13.45 s
Mean throughput	3725.7 samples/s

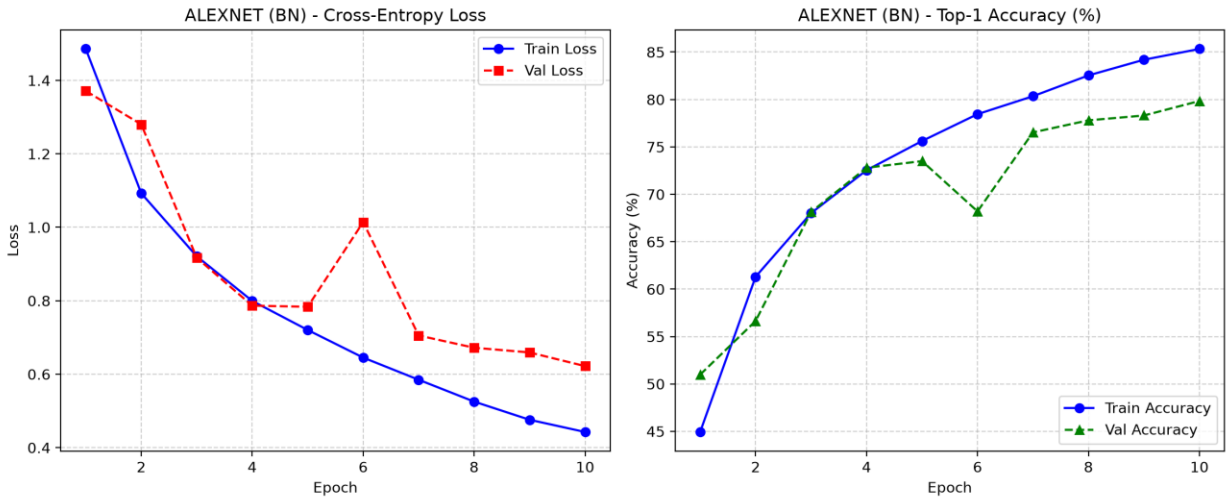


Figure 3: AlexNet-64 + BatchNorm2d training and validation loss and accuracy

Empirical Observations & Questions

Performance Improvement: Did incorporating Batch Normalization yield an improvement in validation accuracy and Macro \(F_1\) over No Normalization and LRN? (Yes / No).

- If Yes: Explain the underlying mathematical and optimization reasons. How does normalizing mini-batch activations reduce internal covariate shift and smooth the optimization landscape?
- If No or Marginal: Discuss what experimental parameters (e.g., choice of learning rate, batch size, or short 10-epoch horizon) may have limited the observable gains.

Yes, incorporating batch normalization yielded an improvement in both validation accuracy and macro F1 over no normalization and LRN. The batch normalization variant resulted in a best validation accuracy of 79.84% and a macro F1 of 80.28%. No normalization reached 73.33% and 73.33%, while the LRN variant reached 73.52% and 73.27%. Relative to no normalization, batch normalization improved accuracy by 6.51 percentage points and Macro F1 by 6.95 percentage points. Relative to LRN, it improved them by 6.32 and 7.01 percentage points.

The mathematical reason is that batch normalization works on one channel at a time. For each channel, it looks at every value that channel produced across the whole mini-batch (all images and all pixel positions). It computes the average (μ) and the spread (σ^2) of those values. It then shifts and scales every value, so the channel's outputs are centered around 0 with a spread of about 1:

$$\hat{x} = \frac{(x - \mu)}{\sqrt{\sigma^2 + \varepsilon}}$$

Here ε is a tiny constant to avoid division by zero. Forcing every channel into the same range could limit what the network is able to learn. To avoid this, batch normalization gives each channel two learnable parameters, a scale (γ) and a shift (β):

$$y = \gamma \cdot \hat{x} + \beta$$

The network adjusts the parameters during training. Each channel therefore starts from a consistent range. Without this as in no normalization or LRN, the range of values each layer outputs keeps shifting as the weights update, a problem known as internal covariate shift. So later layers must constantly readjust, and a single learning rate is too large for some layers and too small for others. By keeping each layer's output in a consistent range, batch normalization reduces internal covariate shift. It also makes the optimization landscape smoother: small weight changes are no longer amplified as they pass through many layers, so the loss changes more predictably and each step at a given learning rate makes more reliable progress and therefore more learning by the end of a fixed epoch window of 10 as seen in this experiment.

Computational Overhead: Did the addition of LRN or BatchNorm2d measurably alter your training epoch wall-clock time and arithmetic FLOPs? (Yes / No). Contrast the computational overhead of element-wise normalization against the convergence benefit.

The addition of LRN and BatchNorm2d did not measurably alter arithmetic FLOPs. BatchNorm2d added around 1.27 MFLOPs (+0.29%) and reported no FLOP difference for LRN. With normalization the average epoch wall-clock time decreased by ~2s with LRN and by ~14s with BatchNorm2d. Since normalization adds computation rather than removing it, this decrease is attributed to inconsistent hardware conditions between runs. However, the expected difference in epoch wall-clock time is to be longer with normalization.

The overhead is small as normalization is elementwise, needing few operations per activation compared to convolutions. For BatchNorm2d, the cost is far outweighed by the convergence benefit. It reached 72.8% validation accuracy in 4 epochs, compared with 8 for no normalization and 9 for LRN. LRN, in contrast, added the largest overhead (epoch wall-clock time) with no accuracy improvement.

Convergence Stability: Compare the variance and smoothness of the training loss trajectories across the three variants. Which normalization scheme demonstrated the most stable gradient progression?

All three variants had smooth training loss trajectories that decreased with every epoch, with no spikes. No normalization and LRN followed similar trajectories while BatchNorm2d descended steeper early in training, reaching 1.49 after the first epoch compared with ~1.80 for the other two. BatchNorm2d stayed below no normalization and LRN and finished at 0.44 compared with 0.69 and 0.68. BatchNorm2d demonstrated the most stable gradient progression as its per-epoch loss reduction shrank consistently every epoch, while no normalization and LRN both showed irregular reductions nearing final epochs.

Step 3: PyTorch Data Augmentation

AlexNet-64 with BatchNorm and Dropout (p = 0.5) was trained on CIFAR-10 for 15 epochs, once without augmentation (--no-augment) and once with the augmentation pipeline (RandomCrop, RandomHorizontalFlip, ColorJitter). All other settings were identical (batch size 128, LR = 0.001, seed 42).

Table 14: Final results with and without data augmentation (15 epochs, CIFAR-10)

Run	Train Acc (%)	Val Acc (%)	Val Macro F1 (%)	Generalization Gap Δ (pp)
Without augmentation	90.47	81.43	81.35	9.04
With augmentation	85.01	82.44	82.46	2.57
Change	−5.46	+1.01	+1.11	−6.47

Table 15: Per-epoch generalization gap with and without data augmentation

Epoch	No Aug Train Acc (%)	No Aug Val Acc (%)	No Aug Δ (pp)	Aug Train Acc (%)	Aug Val Acc (%)	Aug Δ (pp)
1	44.93	50.98	−6.05	42.39	45.10	−2.71
2	61.31	56.64	4.67	57.87	58.04	−0.17
3	68.01	68.14	−0.13	64.69	65.34	−0.65
4	72.54	72.80	−0.26	69.22	70.31	−1.09
5	75.62	73.50	2.12	72.53	71.89	0.64
6	78.45	68.20	10.25	74.80	75.55	−0.75
7	80.34	76.53	3.81	76.75	75.84	0.91
8	82.54	77.80	4.74	78.35	78.45	−0.10
9	84.19	78.31	5.88	79.84	75.86	3.98
10	85.33	79.84	5.49	80.87	79.74	1.13
11	86.19	78.45	7.74	81.58	78.79	2.79
12	87.62	77.34	10.28	82.67	80.02	2.65
13	88.78	80.97	7.81	83.48	81.39	2.09
14	89.66	79.70	9.96	84.14	81.89	2.25
15	90.47	81.43	9.04	85.01	82.44	2.57

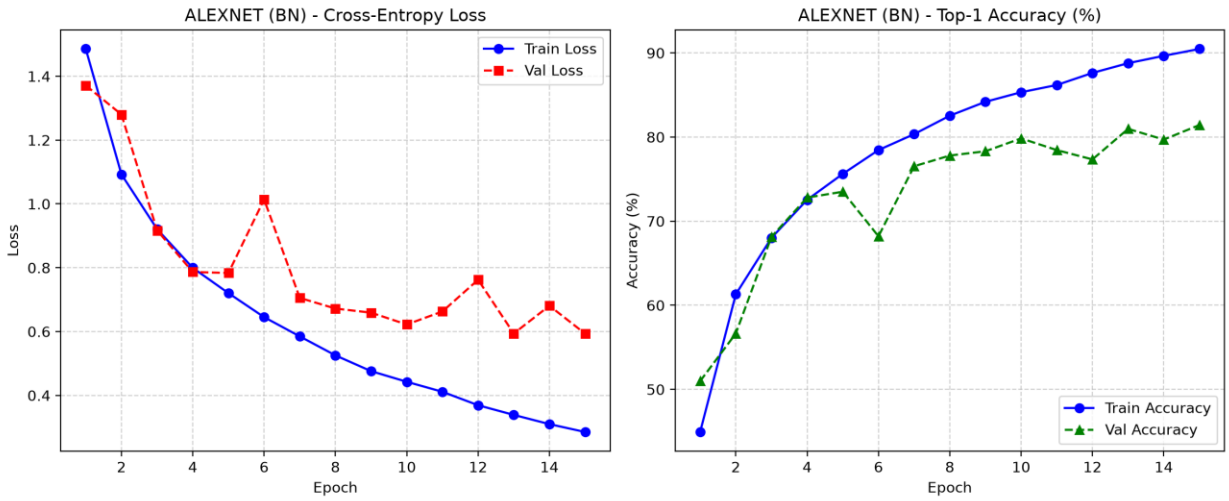


Figure 4: Training curves without data augmentation (15 epochs)

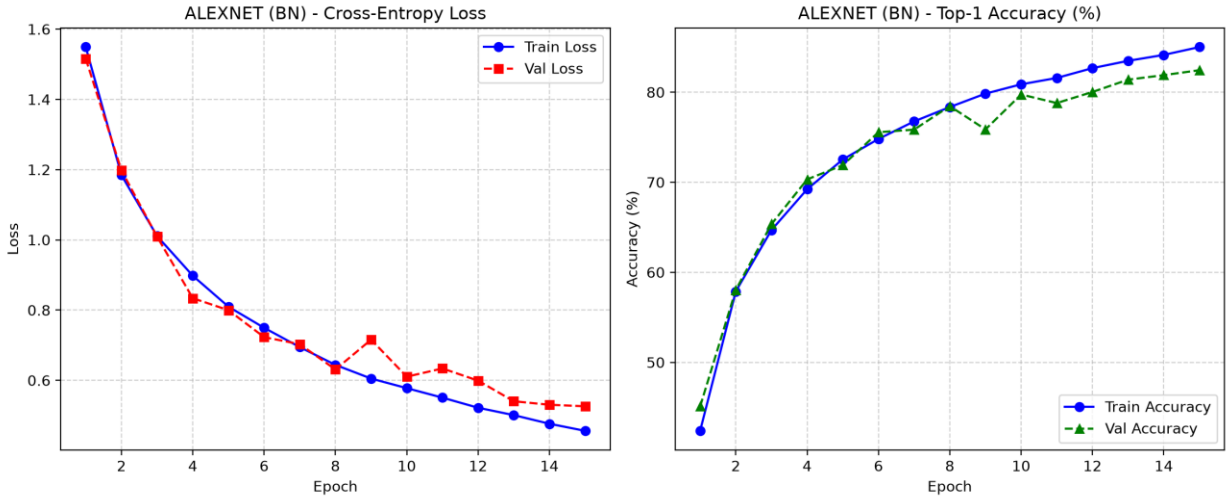


Figure 5: Training curves with data augmentation (15 epochs)

Empirical Questions

Generalization Gap Reduction: Did applying the PyTorch data augmentation pipeline reduce the generalization gap (\Delta)? (Yes / No). Report your measured training accuracy, validation accuracy, and calculated (\Delta) for both the unaugmented run and the augmented run.

Yes, applying the PyTorch data augmentation pipeline greatly reduced the generalization gap. Without augmentation, the model reached a final training accuracy of 90.47% and a validation accuracy of 81.43%, giving a gap of 9.04 percentage points. With augmentation,

the model reached a training accuracy of 85.01% and a validation accuracy of 82.44%, giving a gap of 2.57 percentage points. So, data augmentation reduced the gap by 6.47 percentage points.

Validation Metric Improvement: Did data augmentation lead to an overall increase in final validation accuracy and Macro (F_1)? (Yes / No).

- If Yes: Explain why injecting label-preserving stochastic transformations (random crops, horizontal flips, color jitter) forces convolutional filters to learn invariant semantic features rather than memorizing canonical training sample positions.
- If No or Marginal: Inspect your training curves. Did the network require more epochs to fully assimilate the augmented distribution, or did severe transformations cause underfitting?

Yes, data augmentation led to an overall increase in final validation accuracy and Macro F1. It increased final validation accuracy from 81.43% to 82.44% (+1.01 percentage points) and Macro F1 from 81.35% to 82.46% (+1.11 percentage points).

Each augmentation changes how an image looks without changing what it is. A random crop shifts the object to a slightly different position, a horizontal flip mirrors it, and colour jitter changes its brightness, contrast, and saturation. A flipped, shifted, or slightly darker cat is still a cat, so the label stays correct. These changes are random, so the network sees a slightly different version of every training image each epoch. Without augmentation, the network sees the exact same pixels every epoch and can partly memorize them, for example learning that a certain image has a dark patch in a specific corner. With augmentation, those exact pixel patterns never repeat, so memorizing them doesn't help. The filters are instead pushed to respond to features that stay the same across every version (invariant semantic features), such as the shape of an ear, a wheel, or a wing. Those features also appear in the unseen validation images, which is why the model generalizes better.

Step 4: Dual-Dataset Benchmark

Results

Table 16: AlexNet-64 + BatchNorm2d model complexity on Tiny ImageNet

Metric	Value

Total parameters	6,806,152 (6.806 M)
Trainable parameters	6,806,152
Multiply-accumulates	219.18 MMACs
FLOPs	438.36 MFLOPs
Weight size (FP32)	25.96 MB
Forward/backward pass size	2.56 MB
Estimated total memory	29.83 MB

Table 17: AlexNet-64 + BatchNorm2d layer specification for Tiny ImageNet (200 classes)

Layer	**Output Shape**	**Params**
Conv2d (5×5, s=2, p=2) + BatchNorm2d + ReLU	64 × 32 × 32	4,864 + 128
MaxPool2d (3×3, s=2)	64 × 15 × 15	—
Conv2d (5×5, s=1, p=2) + BatchNorm2d + ReLU	192 × 15 × 15	307,392 + 384
MaxPool2d (3×3, s=2)	192 × 7 × 7	—
Conv2d (3×3, s=1, p=1) + BatchNorm2d + ReLU	384 × 7 × 7	663,936 + 768
Conv2d (3×3, s=1, p=1) + BatchNorm2d + ReLU	384 × 7 × 7	1,327,488 + 768
Conv2d (3×3, s=1, p=1) + BatchNorm2d + ReLU	256 × 7 × 7	884,992 + 512
MaxPool2d (3×3, s=2)	256 × 3 × 3	—
Dropout (0.5) + Linear + ReLU	1,024	2,360,320
Dropout (0.5) + Linear + ReLU	1,024	1,049,600
Linear	200	205,000
Total		6,806,152

Table 18: AlexNet-64 + BatchNorm2d per-epoch results on Tiny ImageNet

Epoch	**Train Loss**	**Train Acc (%)**	**Val Loss**	**Val Acc (%)**	**Val Macro F1 (%)**	**Time (s)**	**Throughput (samples/s)**
1	4.8207	3.44	4.5284	6.05	3.23	36.55	2736.3
2	4.3395	7.47	4.1446	10.04	7.47	35.04	2853.6
3	4.0652	11.15	3.9308	13.06	10.62	34.89	2865.8
4	3.8639	14.24	3.6618	17.63	14.72	35.02	2855.6
5	3.6998	16.84	3.6938	17.22	14.80	34.72	2880.1
6	3.5579	19.23	3.4450	21.65	19.86	34.70	2881.5

7	3.4314	21.45	3.2652	25.03	22.51	34.98	2858.6
8	3.3255	23.28	3.2098	25.73	23.51	34.58	2892.1
9	3.2264	25.32	3.1190	27.65	25.62	35.04	2854.0
10	3.1422	26.77	3.0133	30.02	28.13	34.84	2869.9

Table 19: AlexNet-64 + BatchNorm2d training summary on Tiny ImageNet

Summary	**Value**
Best Val Acc	30.02% (epoch 10)
Best Val Macro F1	28.13% (epoch 10)
Final Train Acc	26.77%
Generalization gap at epoch 10	−3.25 pp
Random-guess baseline	0.5% (1/200)
Total training time	350.83 s
Mean epoch time	35.04 s
Mean throughput	2854.8 samples/s

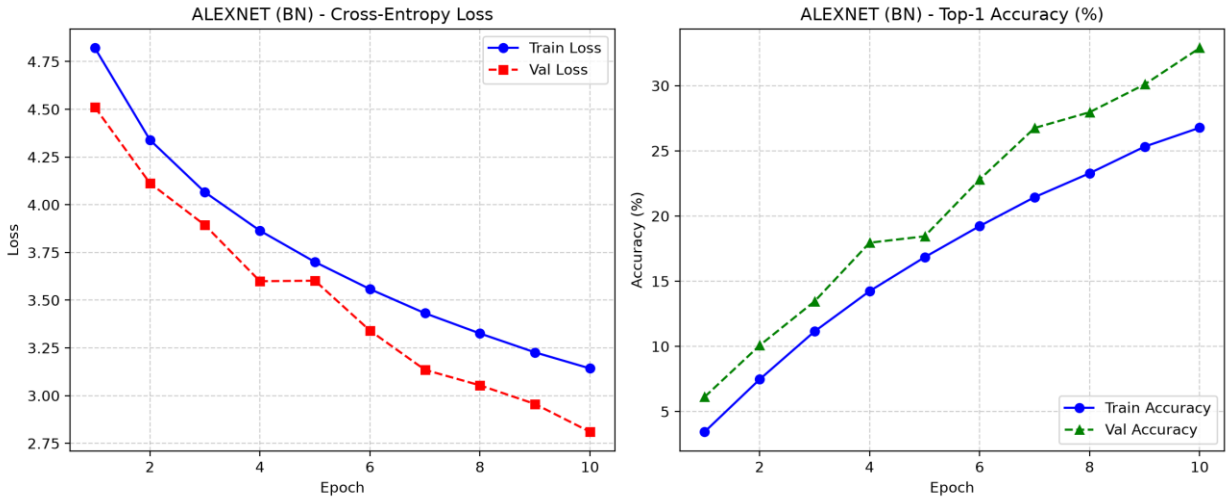


Figure 6: AlexNet-64 + BatchNorm2d training and validation loss and accuracy on Tiny ImageNet

Empirical Questions

Generalization Above Random Chance: Did AlexNet learn discriminative features on Tiny ImageNet significantly above the random guess baseline (\(1/200 = 0.5\%\))? (Yes / No).

Yes. AlexNet reached a best validation accuracy of 30.02% and a Macro F1 of 28.13% on Tiny ImageNet.

Cross-Dataset Performance Comparison: How do your validation accuracy and Macro \(F_1\) on Tiny ImageNet compare to CIFAR-10? Why does Tiny ImageNet present a vastly steeper generalization challenge (considering 200 fine-grained categories, \(64\times 64\) spatial complexity, and reduced training samples per class)?

AlexNet reached 79.74% validation accuracy and 80.00% Macro F1 on CIFAR-10, but only 32.88% and 31.10% on Tiny ImageNet.

Tiny ImageNet presents a much steeper challenge for three reasons. First, it has 200 fine-grained categories, 20 times more than CIFAR-10, some of which are very similar, so the model must learn much finer distinctions. Second, it provides only 500 training images per class compared with 5,000 for CIFAR-10, so the model sees fewer examples of each class to learn from. Third, its 64×64 images are real-world photos with backgrounds and off-centre objects, while CIFAR-10 images are simpler images with no added detail.

Overfitting Trajectory: Did the Tiny ImageNet validation loss diverge or plateau earlier in the training schedule compared to CIFAR-10? Explain this behavior using your generated loss curves.

No. The Tiny ImageNet validation loss decreased steadily throughout training and stayed below the training loss every epoch. In the Tiny ImageNet loss plot, both curves slope downward for all 10 epochs, and the validation loss sits below the training loss the whole time. Neither curve flattens out, so the model was still learning and had not started to overfit.

Step 5: Master Ablation Study Summary Tables, Exploration Menu & Model Profiling

7.1 Results

Table 20: Model complexity benchmark from compare_models.py

Model Architecture	Params (M)	MACs (M)	FLOPs (M)	Latency (ms)	Throughput (img/s)
1. LeNet-5 (Base)	0.338	3.59	7.17	0.15	6834.8
2. LeNet-Modern (ReLU+Pool)	0.446	4.59	9.19	0.15	6463.8
3. AlexNet-64 (None)	6.609	218.35	436.70	0.52	1927.5
4. AlexNet-64 (LRN)	6.609	218.35	436.70	0.64	1552.5

5. AlexNet-64 (BatchNorm)	6.611	218.98	437.97	0.56	1795.1
6. Torchvision Zoo AlexNet	57.045	94.10	188.20	1.18	844.4

7.2 Training Wall Clock Time

Mean wall-clock time per epoch, measured with the CUDA-synchronized Timer in train.py and averaged over three seeds, is reported for every configuration in Table 7.3a. Because some runs were sowed by background load on the laptop GPU, a controlled benchmark was also run: 100 training steps (forward pass, backward pass and Adam update, batch size 128) per model, repeated over three interleaved rounds in one session.

Table 21: Measured training step time and compute per epoch

Model	**Training step, batch 128 (ms)**	**Compute per 390-step epoch (s)**
Base LeNet-5	3.71 ± 0.05	1.45
Modernized LeNet	4.24 ± 0.05	1.65
AlexNet (No Norm)	25.17 ± 0.24	9.82
AlexNet + LRN	38.01 ± 0.20	14.82
AlexNet + BatchNorm	28.45 ± 0.08	11.10
PyTorch Zoo AlexNet	30.47 ± 0.09	11.88

7.3 Master Ablation Study Summary Tables

Table 22: Master ablation table 7.3a, architectural configuration and computational complexity

Model Architecture	**Conv Depth**	**Normalization & Regularization**	**Data Augmentation**	**Trainable Params (M)**	**FLOPs (MFLOPs)**	**Mean Epoch Time (s)**	**Inference Latency (ms)**
1. Base LeNet-5	2	None (Tanh + AvgPool)	None	0.338	7.17	6.84 ± 0.89	0.15
2. Modernized LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.446	9.19	7.58 ± 2.22	0.15
3. AlexNet (No Norm)	5	Dropout (p = 0.5)	None	6.609	436.70	14.96 ± 3.91	0.52

4. AlexNet + LRN	5	LRN + Dropout (p = 0.5)	None	6.609	436.70	21.30 ± 5.52	0.64
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p = 0.5)	None	6.611	437.97	14.39 ± 0.27	0.56
6. AlexNet + BN + Aug	5	BatchNorm2d + Dropout (p = 0.5)	Pure PyTorch Aug	6.611	437.97	16.29 ± 0.01	0.56
7. PyTorch Zoo AlexNet	5	Dropout (p = 0.5), AdaptiveAvgPool2d	None	57.045	188.20	14.72 ± 1.24	1.18

Table 23: Master ablation table 7.3b, predictive performance on CIFAR-10 and Tiny ImageNet

Model Architecture	**CIFAR-10 Val Acc (%)**	**CIFAR-10 Val Macro F1 (%)**	**Tiny ImageNet Val Acc (%)**	**Tiny ImageNet Val Macro F1 (%)**
1. Base LeNet-5	55.85	55.41	12.31	10.32
2. Modernized LeNet	66.27	66.26	19.96	18.24
3. AlexNet (No Norm)	73.33	73.33	0.50	0.00
4. AlexNet + LRN	73.52	73.27	0.50	0.00
5. AlexNet + BatchNorm	79.84	80.28	30.06	28.19
6. AlexNet + BN + Aug	79.74	80.00	30.02	28.13
7. PyTorch Zoo AlexNet	65.98	65.94	10.31	7.93

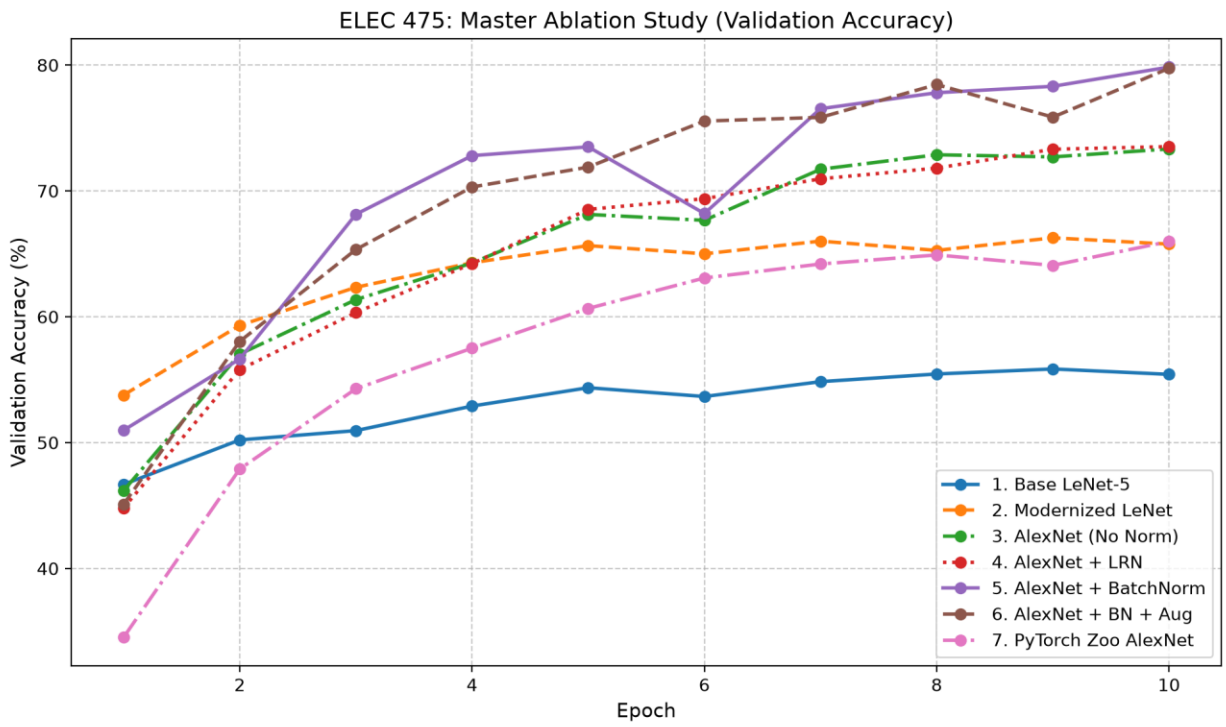


Figure 7: Validation accuracy of all seven models on CIFAR-10 (master ablation study)

Step 6: Post-Lab Analytical Report

Empirical Results & Figures

Master Ablation Study Summary Tables

Done in the above step 5

Extended Ablation Exploration Table

The extended ablation isolates the effect of each design change along the ablation ladder, from Base LeNet-5 to the PyTorch Zoo AlexNet. Each row compares two configurations that differ in one modification and reports Δ = Variant B − Variant A for validation accuracy, validation Macro F1, FLOPs, parameters and generalization gap (final training accuracy minus final validation accuracy).

Table 24: Extended ablation exploration with Δ comparisons

#	Ablation	Comparison	Δ Val Acc (%)	Δ Val Macro F1 (%)	Δ FLOPs (M)	Δ Params (M)	Δ Gen. Gap (pp)

1	Activation and pooling	Base LeNet-5 → Modernized LeNet	+10.42	+10.85	+2.02	+0.108	+0.89
2	Depth and width scaling	Modernized LeNet → AlexNet (No Norm)	+7.06	+7.07	+427.51	+6.163	−11.32
3	Local Response Normalization	AlexNet (No Norm) → AlexNet + LRN	+0.19	−0.06	0.00	0.000	+0.05
4	Batch normalization	AlexNet + LRN → AlexNet + BatchNorm	+6.32	+7.01	+1.27	+0.003	+2.32
4b	Batch normalization (direct)	AlexNet (No Norm) → AlexNet + BatchNorm	+6.51	+6.95	+1.27	+0.003	+2.37
5	Data augmentation (10 epochs)	AlexNet + BN → AlexNet + BN + Aug	−0.10	−0.28	0.00	0.000	−4.36
5b	Data augmentation (15 epochs, Step 3)	AlexNet + BN → AlexNet + BN + Aug	+1.01	+1.11	0.00	0.000	−6.47
6	Model Zoo baseline	AlexNet + BN → PyTorch Zoo AlexNet	−13.86	−14.34	−249.77	+50.434	−2.44
7	Dataset shift	AlexNet + BN on CIFAR-10 → Tiny ImageNet	−49.78	−52.09	+0.39	+0.195	−5.40
7b	Normalization on Tiny ImageNet	AlexNet (No Norm) → AlexNet + BN, Tiny ImageNet	+29.56	+28.19	+1.28	+0.003	—

Multi-seed evaluation results

To check whether the differences above are larger than run-to-run variation, every CIFAR-10 configuration in the ablation ladder was trained three times with seeds 42, 123 and 2026. Seeds were set with seed_everything() in utils/seed.py, with deterministic cuDNN. Values are reported as Mean ± Std, where Std is the sample standard deviation (N − 1), computed with calculate_mean_std() in utils/seed.py. Accuracy and Macro F1 are the best validation

values of each run. The generalization gap is final training accuracy minus final validation accuracy at epoch 10.

Table 25: Multi-seed evaluation results (seeds 42, 123, 2026), Mean ± Std

Model Architecture	**Seed 42 Val Acc (%)**	**Seed 123 Val Acc (%)**	**Seed 2026 Val Acc (%)**	**Val Acc (%) Mean ± Std**	**Val Macro F1 (%) Mean ± Std**	**Gen. Gap (pp) Mean ± Std**	**Mean Epoch Time (s) Mean ± Std**
1. Base LeNet-5	55.85	54.66	53.73	54.75 ± 1.06	54.47 ± 1.00	13.65 ± 0.52	6.84 ± 0.89
2. Modernized LeNet	66.27	66.84	66.52	66.54 ± 0.29	66.46 ± 0.30	14.54 ± 1.61	7.58 ± 2.22
3. AlexNet (No Norm)	73.33	73.97	71.68	72.99 ± 1.18	72.72 ± 1.41	3.56 ± 0.42	14.96 ± 3.91
4. AlexNet + LRN	73.52	75.06	72.04	73.54 ± 1.51	73.25 ± 1.54	2.99 ± 0.23	21.30 ± 5.52
5. AlexNet + BatchNorm	79.84	77.96	79.72	79.17 ± 1.05	79.21 ± 1.39	6.45 ± 1.76	14.39 ± 0.27
6. AlexNet + BN + Aug	79.74	81.28	78.22	79.75 ± 1.53	79.91 ± 1.51	0.86 ± 1.78	16.29 ± 0.01
7. PyTorch Zoo AlexNet	65.98	64.46	65.76	65.40 ± 0.82	65.28 ± 0.81	2.40 ± 1.21	14.72 ± 1.24

Training and validation loss/accuracy curves

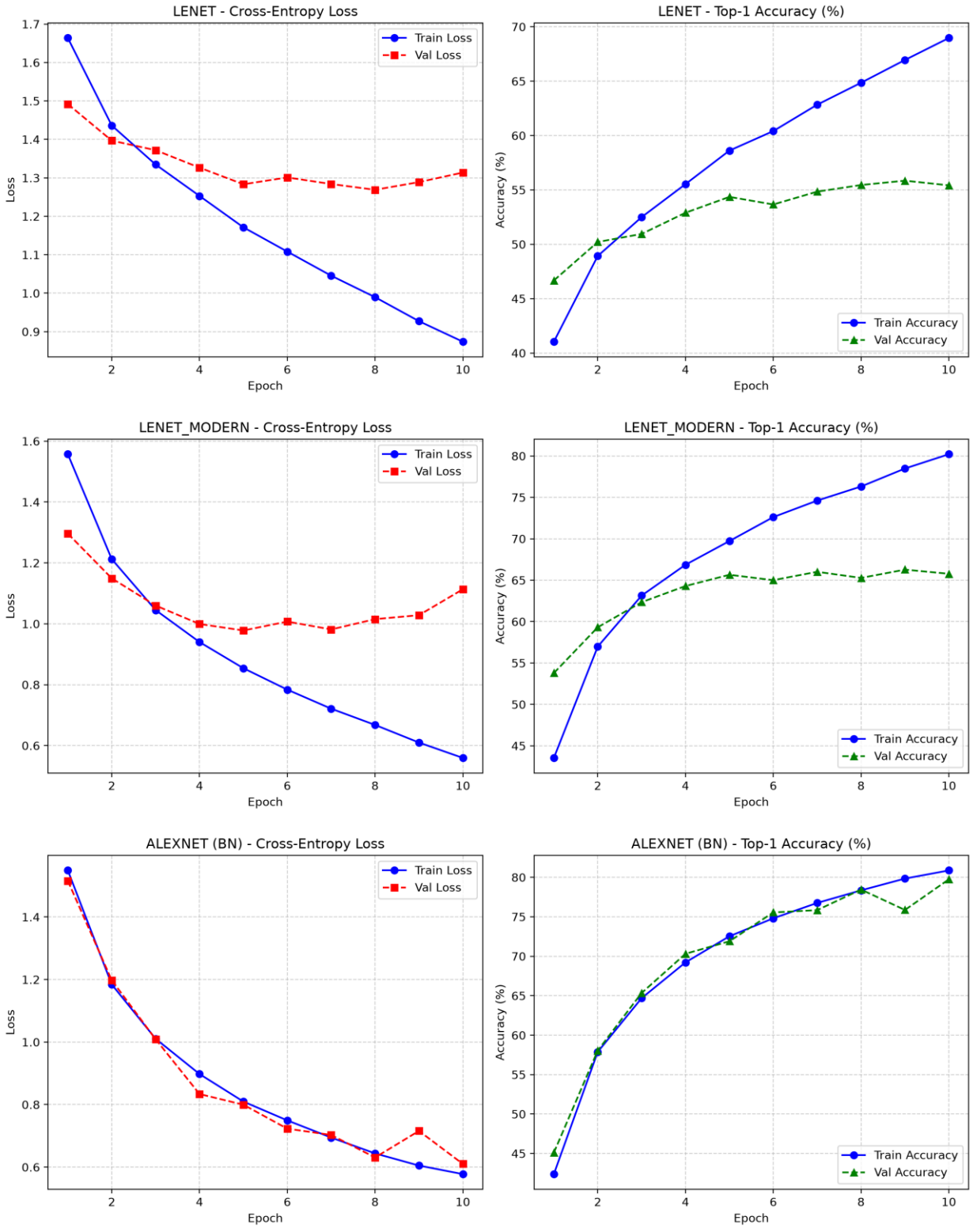


Figure 8: Training and validation curves for Base LeNet-5, Modernized LeNet and AlexNet-64 + BatchNorm2d

Sample output from demo_transforms.py

Table 26: Sample output from demo_transforms.py

Pipeline	Object Type	Data Type	Shape (Layout)	Value Range	Color Space	Memory Location
OpenCV (cv2)	numpy.ndarray	float32	(64, 64, 3) HWC, channels-last	0.0000 to 1.0000	Processed in BGR (OpenCV default), converted BGR → RGB	Host RAM (CPU)
NumPy	numpy.ndarray	float32	(64, 64, 3) HWC, channels-last	0.0000 to 1.0000	RGB	Host RAM (CPU)
PyTorch (torchvision)	torch.Tensor	torch.float32	torch.Size([3, 64, 64]) CHW, channels-first	−1.9895 to 2.1265 (standardized)	RGB	cuda:0 (GPU VRAM)

Benchmark of 1,000 images (Resize + Flip + Normalize):

Table 27: Transform pipeline benchmark on 1,000 images (Resize + Flip + Normalize)

Pipeline	Time (ms)	Throughput (img/s)	Note
OpenCV (CPU)	5.38	185,841	Host-to-device copy still required
NumPy (CPU)	17.18	58,193	Host-to-device copy still required
PyTorch (GPU tensors)	93.16	10,735	No PCIe transfer needed

Class balance and feature distribution

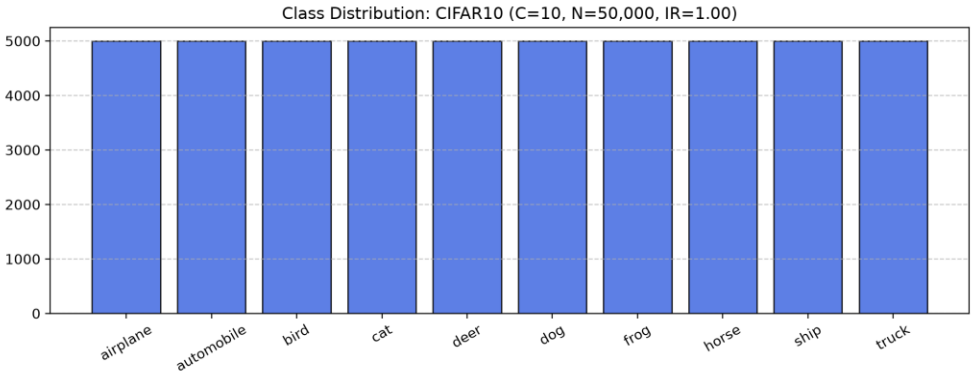


Figure 9: CIFAR-10 class distribution

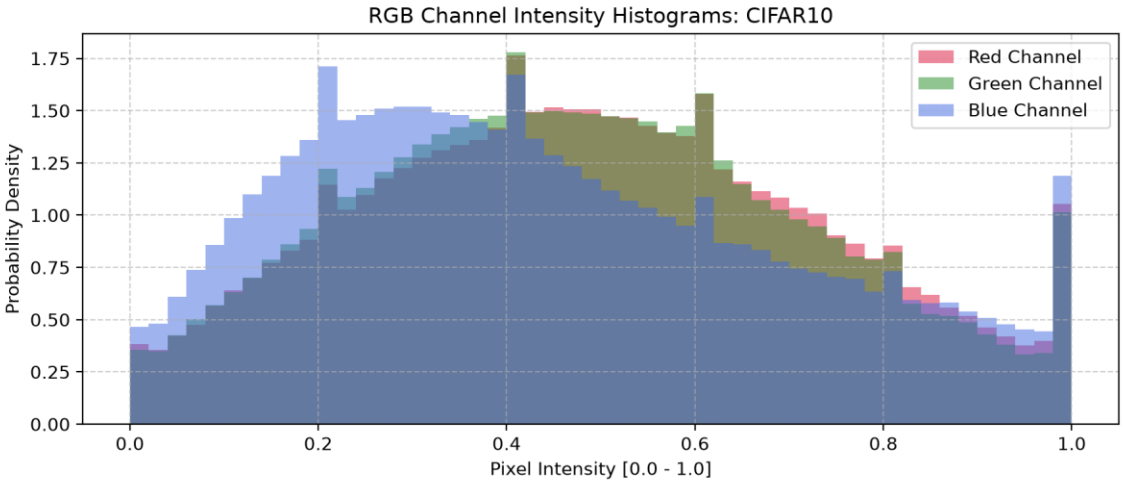


Figure 10: CIFAR-10 RGB channel intensity histograms

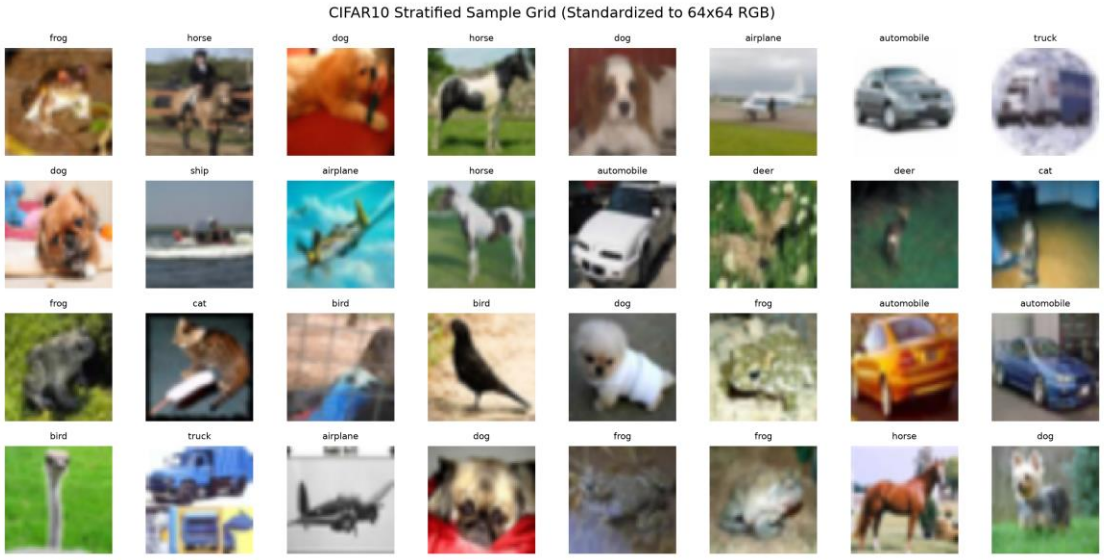


Figure 11: CIFAR-10 stratified sample images

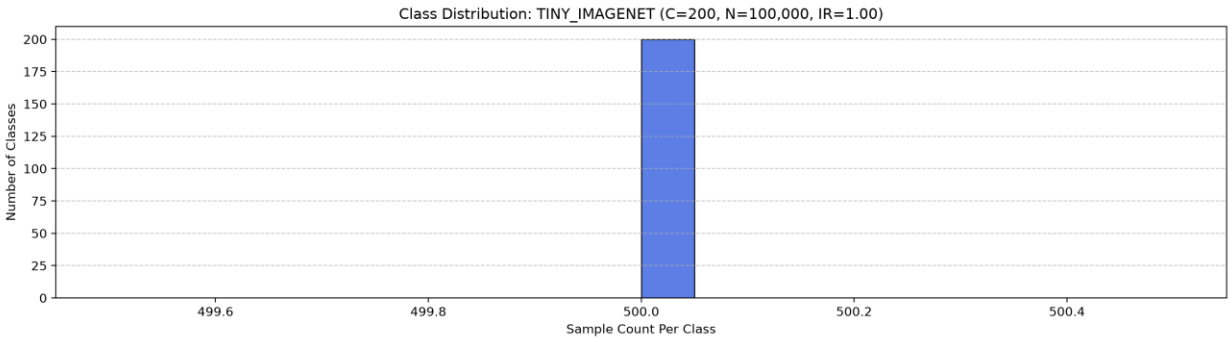


Figure 12: Tiny ImageNet class distribution

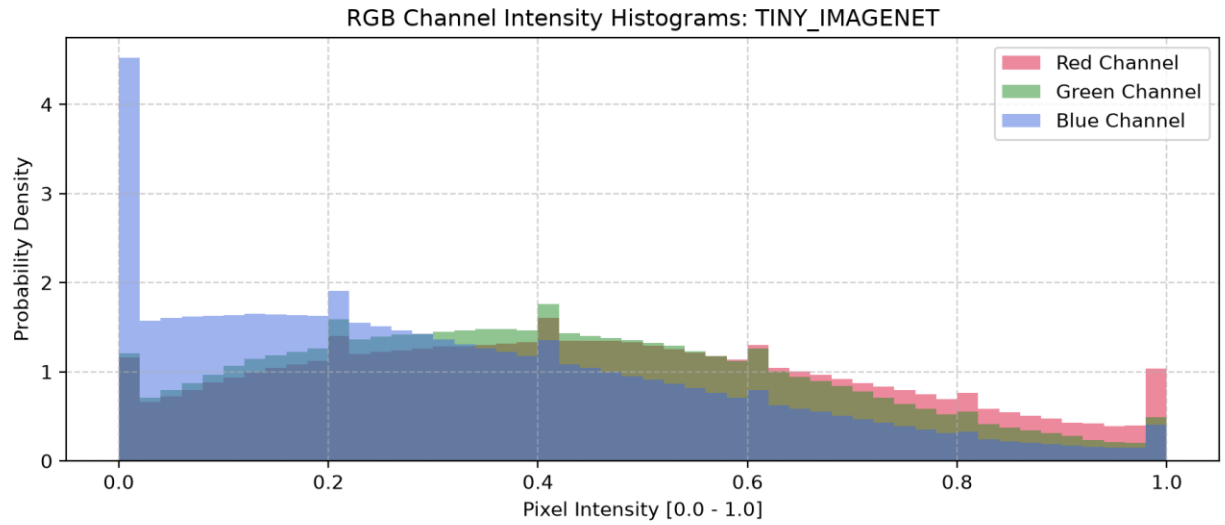


Figure 13: Tiny ImageNet RGB channel intensity histograms

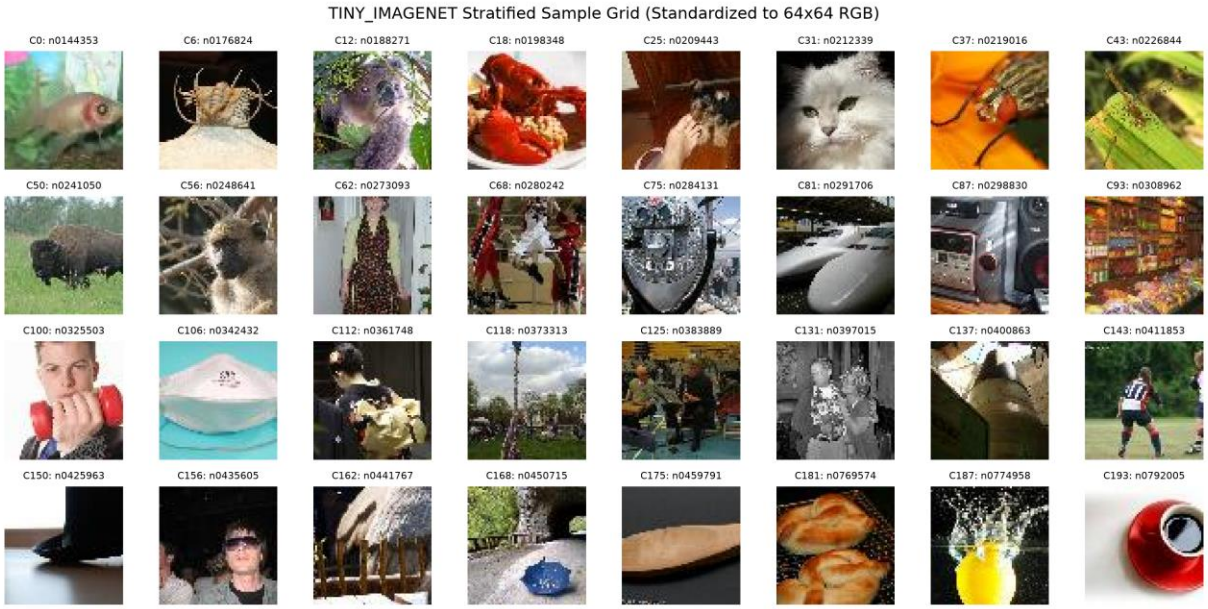


Figure 14: Tiny ImageNet stratified sample images

Terminal output table from compare_models.py

Done in step 5

Technical Analysis & Discussion

Base LeNet-5

Base LeNet-5 is the baseline reaching 55.85% on CIFAR-10 and 12.31% on Tiny ImageNet at 7.17 MFLOPs and 0.15 ms latency. Its Tanh activations saturate weakening gradients and its average pooling blurs features. Its 16-pixel receptive field means each final feature sees only a quarter of the image. It also overfit heavily with a generalization gap of 13.65. Its validation loss rises from about epoch 5 while training loss keeps falling, since the model has no regularization.

Modern LeNet

Replacing Tanh with ReLU and average pooling with overlapping max pooling increased accuracy to 66.27% on CIFAR-10 and 19.96% on Tiny ImageNet, adding only +2 MFLOPs. ReLU does not saturate, providing steeper gradients and faster training. Max pooling keeps the strongest activation in each region rather than averaging it away. However, overfitting was still present with a generalization gap of 14.54.

AlexNet-64 (No Normalization)

Using five convolutional layers with many more channels raised CIFAR-10 accuracy by 7.06 pp to 73.33%. However, it uses about 47 times more computation (436.70 MFLOPs) and 15 times more parameters (6.61 M). With more layers, AlexNet-64's receptive field grows to 97 pixels, allowing each of its final features to see the entire image. The model had overfit the training data much less, reducing the generalization gap to 3.56, likely due to the added dropout (p = 0.5). However, on Tiny ImageNet the model was unsuccessful, reaching only 0.50% accuracy and 0.00% Macro F1. This is likely due to the lack of normalization as the values passed between layers became too large or too small to train on.

AlexNet-64 + LRN

The addition of LRN gave no measurable benefit. Accuracy was 73.54% vs. 72.99% without normalization, and it also was unsuccessful on Tiny ImageNet (0.50% accuracy). At the same time, it was the most expensive normalization. Mean epoch time rose from 14.96s to 21.30s and latency from 0.52 to 0.64 ms.

AlexNet-64 + BatchNorm2d

BatchNorm2d was the single most effective change in the lab. On CIFAR-10 it reached 79.84% accuracy and on Tiny ImageNet it reached 30.06%. It came at a low cost of 1.27 MFLOPs more, with a mean epoch time of 14.39, similar to the no normalization time. By keeping each layer's values within a range, BatchNorm2d makes training smoother and each update makes steady progress. As a result, it reached ~70% accuracy in just 4 epochs, compared with around 8 for the other variants. Generalization gap was 6.45, larger than other variants as it learns the training data faster and begins to overfit sooner.

AlexNet-64 + BatchNorm2d + Augmentation

Data augmentation consistently reduced overfitting, but its effect on accuracy was small. Generalization gap fell from 6.45 to 0.86 across seeds, and from 9.04 to 2.57 in the 15-epoch run. However, accuracy increased only from 79.17% to 79.75% at 10 epochs and on Tiny ImageNet, accuracy was essentially unchanged. Random crops, flips and colour jitter stop the model from memorizing pixel patterns, but they also make training harder. More epochs are needed before the better generalization turns into higher accuracy. Augmentation also raised the mean epoch time from 14.39s to 16.29s, since every image must be transformed during training.

PyTorch Model Zoo AlexNet

Despite having the most parameters (57.04 M), the Zoo AlexNet model reached only 65.98% on CIFAR-10, below AlexNet without normalization. On Tiny ImageNet it reached

10.31% and had the highest latency (1.18 ms). This model was designed for much larger images (244x244). In our 64×64 images, its first layers reduce the image down to a single pixel per channel, losing most detail about the location of features in the image making it more difficult to train. This shows that the model must be designed for the size of its input images.